

Clasificacion de textos segun los ODS

Guia de estudio: teoria, decisiones y preguntas de sustentacion

Machine Learning No Supervisado - Maestria en Inteligencia Artificial

Microproyecto 2: bolsa de palabras + TF-IDF, LSA (SVD truncado) y clasificacion supervisada

Que es este documento. No es uno de los dos entregables del curso (esos son el notebook y `streamlit_app.py`). Es material de repaso personal: resume que se hizo, por que se hizo asi y la teoria detras de cada decision, en el orden en que probablemente se pregunte en la sustentacion.

1. Resumen ejecutivo

El proyecto construye un clasificador de texto en español que predice a cual Objetivo de Desarrollo Sostenible (ODS) pertenece un parrafo. Es un problema supervisado de clasificacion multiclase, pero el curso exige resolverlo usando tecnicas no supervisadas de representacion de texto: bolsa de palabras ponderada con TF-IDF y analisis semantico latente (LSA) mediante SVD truncado.

El pipeline final, de principio a fin, es:

texto crudo -> limpieza (clean_text) -> TF-IDF (1-2 gramas) -> TruncatedSVD (300 componentes) -> Normalizer L2 -> LinearSVC (C=0.3, class_weight=balanced)

Resultado en el conjunto de test (1 932 textos nunca vistos por el modelo):

Metrica	Valor
Accuracy	0.885
Precision macro	0.853
Recall macro	0.855
F1 macro (metrica principal)	0.854
F1 ponderado	0.885

Todo el trabajo se organizo en un notebook de Jupyter unico y autocontenido (13 secciones), mas una app de Streamlit (bono) que reutiliza exactamente el mismo pipeline serializado. Las siguientes secciones explican la teoria detras de cada bloque del pipeline en el orden en que aparece, y despues resumen las decisiones tomadas y las preguntas que probablemente surjan en la sustentacion.

2. El problema y los datos

2.1 Por que es un problema de clasificacion multiclase

Cada texto tiene una unica etiqueta ODS (un numero entre 1 y 16). El objetivo es aprender una funcion que, dado un texto nuevo, devuelva esa etiqueta. Es **supervisado** porque el dataset trae la etiqueta correcta para entrenar; es **multiclase** porque hay mas de dos categorias posibles y cada texto pertenece a exactamente una.

2.2 El dataset

- 9 656 textos en español, columnas textos y ODS, sin nulos ni duplicados.
- Proviene del OSDG Community Dataset (2023), traducidos al español con DeepL y aumentados con la API de ChatGPT.
- Textos de tipo informe/articulo, longitud media ~110 palabras (mediana ~105, rango 24-268).
- **Solo aparecen las etiquetas ODS 1 a 16.** El ODS 17 (Alianzas para lograr los objetivos) no tiene ejemplos: el modelo no puede aprenderlo ni predecirlo. Esto se verifica en la auditoria inicial del notebook (seccion 3) y se repite como advertencia en la app.
- Desbalance moderado: de 312 textos (ODS 12) a 1 080 (ODS 16), razon 3.5:1 entre la clase mas grande y la mas pequena.

Estos dos hechos (ODS 17 ausente y desbalance moderado) son la raiz de varias decisiones posteriores: la metrica principal (F1 macro), el uso de `class_weight="balanced"` y la advertencia explicita en la app de que solo existen 16 clases.

3. Teoria: limpieza de texto, bolsa de palabras y TF-IDF

3.1 Bolsa de palabras (Bag of Words)

Un modelo de ML no puede recibir texto directamente: necesita numeros. La bolsa de palabras representa cada documento como un vector donde cada posicion corresponde a un termino del vocabulario y el valor mide cuanto aparece ese termino en el documento. Se llama "bolsa" porque se pierde el orden de las palabras: solo importa que aparecen y cuantas veces.

3.2 Ponderacion TF-IDF

Contar palabras a secas favorece a las que son frecuentes en todo el idioma (articulos, preposiciones) y no discriminan nada. TF-IDF corrige esto multiplicando dos factores por cada termino t en un documento d :

- **TF (term frequency):** cuantas veces aparece t en d . En este proyecto se usa la variante `sublinear_tf=True`, que aplica $1 + \log(\text{tf})$ en vez de tf puro, para que repetir una palabra 10 veces no la haga 10 veces mas importante (rendimientos decrecientes).
- **IDF (inverse document frequency):** $\log(N / \text{df}(t))$, donde N es el numero de documentos y $\text{df}(t)$ el numero de documentos que contienen t . Un termino que aparece en casi todos los documentos tiene IDF cercano a 0 (poco informativo); uno raro tiene IDF alto.
- El peso final es $\text{TF} \times \text{IDF}$: alto solo cuando el termino es frecuente en ese documento pero raro en el corpus general, exactamente el patron de una palabra caracteristica de un ODS (por ejemplo "reforestacion" para ODS 15).

Cada vector documento se normaliza despues con norma L2 (norma euclidiana = 1), por defecto en `TfidfVectorizer`. Esto hace que la representacion sea independiente de la longitud del texto: un parrafo largo y uno corto sobre el mismo tema quedan comparables.

3.3 Decisiones de limpieza (funcion `clean_text`)

La limpieza se implementa como una funcion y se inyecta como preprocessor del `TfidfVectorizer`. Esto es clave: al vivir *dentro* del vectorizador, la limpieza se vuelve parte del pipeline de scikit-learn y se aplica identica en entrenamiento, validacion cruzada, test y en la app (no hay ningun paso manual que se pueda hacer distinto en cada lugar).

Decision	Justificacion
Minusculas	"Agua" y "agua" deben tratarse como el mismo termino.
Eliminar URLs	No aportan significado semantico y generan tokens unicos que no generalizan.
Normalizar Unicode a NFC	Evita que una misma vocal acentuada tenga dos codificaciones binarias distintas.
Eliminar digitos y puntuacion	Años, porcentajes y simbolos no distinguen ODS y solo agrandan el vocabulario.
Conservar tildes y ñe (n con virgulilla)	Quitarlas fusionaria palabras distintas (termino/termino sin tilde vs. termino) y afea los topicos del LSA.
Stopwords: NLTK espanol + ~25 terminos de dominio	Palabras funcionales y verbos genericos ("puede", "ademas", "ejemplo") aparecen en todos los ODS por igual: son ruido, no senal.

Decision	Justificacion
Sin stemming ni lematizacion	El corpus es formal y traducido de forma homogenea; mantener palabras completas conserva legibles los terminos del LSA y los coeficientes del clasificador. Se prioriza interpretabilidad sobre una ganancia marginal de generalizacion.
Tokens de al menos 3 letras	Elimina residuos de una o dos letras que sobreviven a la limpieza.

3.4 Parametros del TfidfVectorizer

Parametro	Valor	Por que
ngram_range	(1, 2)	Los bigramas capturan expresiones fijas muy discriminantes: "cambio climatico", "salud mental", "derechos humanos".
min_df	3	Un termino en menos de 3 documentos es casi seguro ruido (nombres propios, errores de traduccion) y no generaliza.
max_df	0.90	Elimina terminos presentes en mas del 90% de los textos (stopwords residuales).
sublinear_tf	True	Usa $1 + \log(\text{tf})$ en vez de tf puro (ver 3.2).
max_features	30 000	Acota memoria y tiempo del grid search sin perder terminos relevantes.
norm	L2 (por defecto)	Cada documento queda con norma 1, independiente de su longitud.

Con estos parametros, la matriz TF-IDF de entrenamiento queda en 7 724 documentos x ~26 000 terminos, con densidad menor al 1% (casi todos los valores son cero). Este espacio disperso y de muy alta dimension es exactamente el problema que resuelve la reduccion de dimensionalidad de la seccion siguiente.

4. Teoria: LSA con SVD truncado

4.1 Que es el Analisis Semantico Latente (LSA)

LSA aplica una descomposicion en valores singulares (SVD) truncada a la matriz TF-IDF X (documentos x terminos):

$$X \text{ aprox. } U_k * S_k * V_k^T$$

donde k es el numero de componentes que se conservan (mucho menor que el numero de terminos). Cada fila de V_k^T (una **componente**) es una combinacion lineal de terminos originales: los terminos con mayor peso (en valor absoluto) en esa componente describen un "topico" latente. Cada documento queda representado por sus coordenadas en esas k direcciones, en vez de por sus ~26 000 frecuencias de termino.

4.2 Por que TruncatedSVD y no PCA

PCA centra los datos (resta la media) antes de proyectar, lo que convierte la matriz dispersa en una matriz densa: con 26 000 columnas eso es inviable en memoria. TruncatedSVD opera directamente sobre la matriz dispersa sin centrarla, por lo que es la herramienta estandar para reducir dimensionalidad sobre representaciones TF-IDF. Cuando se aplica sobre una matriz de terminos-documentos, este procedimiento se llama especificamente LSA.

4.3 Dos usos distintos de la misma tecnica

El proyecto usa SVD truncado dos veces, con proposito y numero de componentes distintos, y esto es una de las decisiones que mas conviene poder explicar en la sustentacion:

Uso	k (componentes)	Objetivo	Resultado
LSA interpretativo (Actividad 2, seccion 7)	15 (rango exigido: 10-20)	Que las componentes sean legibles como topicos y se puedan asociar a un ODS	F1 macro de referencia = 0.715 (insuficiente para clasificar bien)
SVD del clasificador (Actividad 3, seccion 8-9)	100 / 200 / 300 (hiperparametro buscado en grid)	Maxima capacidad discriminante para separar 16 clases	F1 macro CV = 0.849 con 300 componentes

La razon de la diferencia: con pocas componentes los ejes son claros y faciles de nombrar, pero se pierde granularidad; con muchas se recupera capacidad de separacion pero los ejes dejan de ser interpretables uno por uno. El notebook demuestra esto con dos referencias numericas (15 componentes vs. sin reduccion, F1 = 0.870) antes de decidir el rango de busqueda {100, 200, 300} para el clasificador.

4.4 Por que la varianza explicada es tan baja (~3%) y por que no importa

En una matriz TF-IDF dispersa, de 26 000 dimensiones y con cada fila normalizada, la varianza total esta repartida entre miles de direcciones casi ortogonales. Que 15 componentes expliquen solo ~3% de la varianza es normal y no invalida el analisis: en LSA lo relevante no es cuanta varianza capturan las primeras componentes, sino si esas direcciones corresponden a una estructura semantica reconocible. Eso se verifica mirando (a) los terminos con mayor peso en cada componente y (b) la puntuacion media de la componente por ODS real (una validacion cuantitativa que usa las etiquetas solo para comprobar la interpretacion, no para ajustar el LSA, que sigue siendo no supervisado).

4.5 Los 15 componentes interpretados en el proyecto

Comp.	Terminos principales	ODS asociado	Evidencia
C1	países, desarrollo, mujeres, políticas, agua, educación	Eje generico (sin valor discriminante)	Medias parejas (~0.11) en varios ODS
C2	derechos, derecho internacional, artículo, ley, justicia	ODS 16 - Paz, justicia e instituciones	Media 0.129 en ODS 16 vs. <=0.03 en el resto
C4	estudiantes, educación, escuelas, docentes, primaria	ODS 4 - Educación de calidad	Media 0.093 en ODS 4
C5	mujeres, género, hombres, igualdad, participación	ODS 5 - Igualdad de género	Media 0.069 en ODS 5
C6	salud, atención, salud mental, atención primaria	ODS 3 - Salud y bienestar	Media 0.073 en ODS 3
C7	agua, aguas subterráneas, aguas residuales, agua potable	ODS 6 - Agua limpia y saneamiento	Media 0.112 en ODS 6
C8	energía, electricidad, renovables (polo -: cambio climático)	ODS 7 - Energía asequible	Media 0.063 en ODS 7
C9	países OCDE, cambio climático (polo -: pobreza, hogares)	ODS 13 - Acción por el clima	Media 0.062 en ODS 13
C10	(polo -) empleo, laboral, trabajadores / (+) pobreza infantil	ODS 8 (polo -) y ODS 1 (polo +)	Media 0.056 en ODS 1
C11	transporte, ciudades, zonas rurales, público	ODS 11 - Ciudades / ODS 9 - Infraestructura	Medias 0.042 y 0.039
C13	producción, alimentos, pesca, seguridad alimentaria	ODS 2 - Hambre cero (también ODS 14, 12)	Medias 0.067 / 0.054 / 0.046

Conclusión del LSA: al menos 8 de las 15 componentes se asocian con claridad a un ODS concreto (cumple el mínimo de 5 exigido por la rúbrica); otras capturan contrastes entre familias de ODS (social vs. ambiental, laboral vs. pobreza). Los ODS con vocabulario compartido (1/10, 9/11, 12/14/15) aparecen mezclados en las mismas componentes, lo que anticipa exactamente las confusiones que después comete el clasificador (sección 8 de este documento).

5. Teoria: el pipeline de clasificacion

5.1 Arquitectura completa

`TfidfVectorizer(preprocessor=clean_text, 1-2 gramas) -> TruncatedSVD(n_components=k) -> Normalizer(norm='l2') -> clasificador lineal (class_weight='balanced')`

Todo vive dentro de un unico Pipeline de scikit-learn. Esto garantiza dos cosas: (1) en cada fold de la validacion cruzada el vocabulario, el IDF y el SVD se reajustan solo con los datos de entrenamiento de ese fold, sin fuga de informacion hacia el fold de validacion; (2) el objeto que se serializa con `joblib` para la app es exactamente el mismo que se evaluo.

5.2 Por que Normalizer(L2) despues del SVD

Es un paso clasico de LSA: tras proyectar al espacio latente, cada documento se vuelve a normalizar a norma 1. Esto hace que el clasificador lineal compare documentos por similitud coseno (la medida natural en LSA) y evita que la distinta escala de cada componente sesgue el aprendizaje.

5.3 Por que reduccion de dimensionalidad antes de clasificar

- Trabaja directamente sobre la matriz TF-IDF dispersa (no hay que densificar 26 000 columnas).
- Proyecta a un espacio denso y semantico: dos textos con vocabulario relacionado pero no identico ("escuela"/"docentes"/"alumnos") quedan cerca, lo que ayuda a generalizar.
- Reduce el ruido de terminos raros y acelera el entrenamiento del clasificador.

5.4 Por que clasificadores lineales (y no arboles, boosting o redes neuronales)

Candidato	Idea del algoritmo	Por que se considero	Hiperparametro
LogisticRegression (multinomial)	Modela $P(\text{clase} x)$ ajustando pesos que maximizan la verosimilitud; frontera de decision lineal.	Produce probabilidades calibradas por clase, utiles para un ranking "top-3"; entrena rapido; regularizacion L2 controla el sobreajuste.	C (inverso de la fuerza de regularizacion)
LinearSVC (SVM lineal)	Busca el hiperplano que separa las clases maximizando el margen (distancia al punto mas cercano de cada clase).	Clasificador de referencia historico para texto con TF-IDF/LSA; muy eficiente con miles de rasgos; robusto a rasgos correlacionados.	C (penaliza cion por violar el margen)

Razones para restringirse a modelos **lineales**: (i) en espacios de texto de alta dimension las clases suelen ser casi linealmente separables, y modelos no lineales (arboles, boosting, kernels RBF) rara vez mejoran el resultado y cuestan ordenes de magnitud mas tiempo de computo; (ii) sus coeficientes son interpretables; (iii) con ~7 700 documentos de entrenamiento el riesgo de sobreajuste se controla bien solo con el hiperparametro C. No se consideraron redes neuronales o transformers porque el enunciado pide expresamente una solucion basada en bolsa de palabras + TF-IDF + reduccion de dimensionalidad.

5.5 El hiperparametro C (regularizacion)

En ambos algoritmos, C controla el balance entre ajustar bien los datos de entrenamiento y mantener los pesos del modelo pequenos (regularizacion). Un C grande penaliza poco los errores de regularizacion y el modelo puede sobreajustar componentes ruidosas del SVD; un C pequeno fuerza pesos mas chicos y un modelo mas simple. En este proyecto gano el C mas pequeno explorado en ambos algoritmos, senal de que

conviene regularizar con fuerza dado el numero de dimensiones (100-300) frente al numero de documentos de entrenamiento.

5.6 Por que `class_weight='balanced'`

Multiplica el termino de perdida de cada clase por un peso inversamente proporcional a su frecuencia, de modo que las clases pequenas (ODS 12, 15, 9) pesen tanto como las grandes (ODS 16, 5, 4) durante el entrenamiento, sin necesidad de sobremuestrear ni submuestrear datos. Es la respuesta directa al desbalance moderado detectado en el analisis exploratorio.

6. Teoria: validacion cruzada, busqueda de hiperparametros y metricas

6.1 Por que separar train/test antes de ajustar nada

El split (80% train / 20% test, estratificado, random_state=42) se hace en la seccion 5 del notebook, **antes** de ajustar el vocabulario TF-IDF, el SVD o cualquier clasificador. Si el vocabulario o el IDF se calcularan con todo el dataset (incluido el test), informacion del test se filtraria al entrenamiento ("data leakage") y las metricas finales quedarian optimistas, sin reflejar el desempeno real sobre datos nuevos. El test se usa una sola vez, al final (secciones 10-11), y nunca para elegir modelo, hiperparametros o preprocesamiento.

6.2 Estratificacion

stratify=y asegura que cada ODS mantenga la misma proporcion en train y test (y, mas adelante, en cada fold de la validacion cruzada). Sin esto, con clases de apenas 312-343 textos, un split al azar podria dejar muy pocos ejemplos de una clase en test y hacer inestables sus metricas.

6.3 Validacion cruzada estratificada y GridSearchCV

La busqueda de hiperparametros usa GridSearchCV con StratifiedKFold(5, shuffle=True, random_state=42) sobre el conjunto de entrenamiento unicamente. Esto entrena y evalua cada combinacion de hiperparametros 5 veces (una por fold), reajustando el pipeline completo (TF-IDF, SVD y clasificador) solo con los 4 folds de entrenamiento de esa iteracion, y promedia la metrica sobre el fold de validacion restante. El resultado es una estimacion mas estable y menos optimista que un unico split train/validation.

Rejilla explorada (90 ajustes en total):

Modelo	svd__n_components	clf__C	Ajustes (x5 folds)
LogisticRegression	100, 200, 300	1, 10, 30	45
LinearSVC	100, 200, 300	0.3, 1, 3	45

C se explora en escala logaritmica (lo habitual para regularizacion) alrededor de los valores por defecto de cada algoritmo; el numero de componentes se explora en un rango amplio porque la seccion 4.3 de este documento (seccion 8 del notebook) ya habia mostrado que 15 componentes son insuficientes para separar 16 clases.

6.4 Por que F1 macro es la metrica principal (y no accuracy)

Con clases desbalanceadas, la accuracy (proporcion de aciertos) puede ser alta con solo acertar en las clases grandes e ignorar las pequenas. F1 macro promedia el F1 de **cada clase por separado, con el mismo peso**, sin importar cuantos ejemplos tenga:

F1 macro = (1/C) * suma de F1_c para c = 1 hasta C, con F1_c = 2 * precision_c * recall_c / (precision_c + recall_c)

Esto refleja el criterio real del proyecto: para un organismo como el UNFPA, clasificar bien un texto de ODS 12 (poco frecuente) importa tanto como uno de ODS 16 (muy frecuente). Se reportan ademas, como complemento, accuracy, precision/recall macro y F1 ponderado (que pondera cada clase por su soporte y por eso se parece mas a la accuracy).

6.5 Matriz de confusion y pares mas confundidos

La matriz de confusion (normalizada por fila) muestra, para cada ODS real, en que otros ODS se reparten sus predicciones. Permite distinguir errores aleatorios de errores sistematicos: en este proyecto los errores se concentran en ODS con vocabulario compartido (ver seccion 8), lo cual es informacion mas util que un solo numero global.

6.6 Por que LinearSVC no da probabilidades

LinearSVC optimiza el margen geometrico entre clases, no una verosimilitud, por lo que no expone `predict_proba`. Para mostrar un ranking de los ODS mas probables (en la evidencia de test y en la app), se usa `decision_function`, que da un margen de decision por clase. El proyecto es explicito en llamarlo "puntuacion relativa" o "margen" y no "probabilidad", para no sugerir una calibracion que el modelo no tiene.

7. Que se hizo, paso a paso (mapa del notebook)

Seccion	Contenido	Idea clave
1-2	Contexto del problema, alcance real (ODS 1-16), reproducibilidad	RANDOM_STATE=42 en split, CV, SVD y clasificadores
3	Auditoria: shape, nulos, duplicados, etiquetas presentes	9 656 filas, 0 nulos, 0 duplicados, falta ODS 17
4	EDA: distribucion por clase y longitud de textos	Desbalance moderado -> F1 macro + class_weight
5	train_test_split(test_size=0.2, stratify=y, random_state=42)	El test queda aislado desde aqui
6	Limpieza (clean_text) + TfidfVectorizer, con tabla de justificacion de cada decision	Actividad 1 del enunciado
7	LSA: comparacion k=10/15/20, se fija k=15, interpretacion de 12 componentes vs. ODS	Actividad 2 del enunciado
8	Arquitectura del pipeline completo; justificacion de SVD, Normalizer, algoritmos y metricas; referencias SVD-15 y sin SVD	Contexto para el grid search
9	GridSearchCV (90 ajustes), tabla y grafico F1 vs. componentes, seleccion de LinearSVC	Actividad 3 del enunciado
10	Evaluacion unica en test: metricas globales, reporte por clase, matriz de confusion, pares confundidos	Actividad 4 del enunciado
11	8 textos aleatorios de test + 5 errores informativos, con top-3 por margen	Evidencia sobre texto no visto (minimo 4 exigidos)
12	Conclusiones, limitaciones y mejoras futuras	Sintesis critica del proyecto
13	joblib.dump del pipeline final + verificacion de recarga + prediccion de 4 textos nuevos	Bono: base para la app Streamlit

8. Resultados finales y su lectura

8.1 Tendencias observadas en la busqueda de hiperparametros

- Mas componentes SVD siempre mejora el F1 macro, de forma monotona en ambos algoritmos (100 -> 200 -> 300), pero con rendimientos decrecientes: con 300 se llega a ~2 puntos de la referencia sin reduccion (0.870), es decir, 300 dimensiones densas retienen casi toda la informacion discriminante de las ~26 000 dispersas (una reduccion de ~99%).
- Gana siempre el C mas pequeno explorado en cada algoritmo: mas regularizacion generaliza mejor sobre las componentes del SVD, que son mas ruidosas cuantas mas se usan.
- LinearSVC (C=0.3, 300 componentes) supera a LogisticRegression (C=1, 300 componentes): F1 macro CV 0.849 +/- 0.006 frente a 0.844 +/- 0.004, y entrena aproximadamente el doble de rapido (~77 s vs. ~165 s por ajuste). La diferencia es pequena pero consistente en todas las configuraciones equivalentes.

8.2 Metricas en el test aislado

F1 macro = 0.854 y accuracy = 0.885 en test, muy cercanas a las de validacion cruzada (0.849 / 0.874): el modelo no esta sobreajustado al procedimiento de seleccion de hiperparametros, y la estimacion por CV

resultado fiable. Precision macro (0.853) y recall macro (0.855) son casi iguales, lo que confirma que `class_weight='balanced'` evita que el modelo prefiera sistematicamente las clases grandes.

8.3 Clases faciles vs. clases dificiles

Grupo	ODS	F1	Por que
Mejor desempeno	16 Justicia, 4 Educacion, 5 Genero, 3 Salud, 6 Agua, 7 Energia, 14 Vida submarina	> 0.90	Vocabulario propio y poco compartido con otros ODS (coincide con las componentes LSA mas nitidas)
Peor desempeno	8 Trabajo decente (0.591), 10 Desigualdades (0.676), 9 Infraestructura (0.739)	< 0.75	Vocabulario fuertemente compartido con otros ODS (empleo/ingresos/crecimiento aparece en 1, 8, 9 y 10) y soporte medio-bajo

La matriz de confusion muestra que los errores no son aleatorios: se concentran en el bloque tematico ODS 8 (Trabajo) <-> ODS 10 (Desigualdades) <-> ODS 1 (Pobreza), seguido de ODS 9 -> ODS 8, ODS 15 -> ODS 13 y ODS 5 -> ODS 8. Son exactamente las familias de ODS que el LSA ya agrupaba en una misma componente o en polos de un mismo contraste (seccion 4.5). Muchos de esos textos serian ambiguos incluso para un anotador humano: un texto sobre empleo de personas en situacion de pobreza puede razonablemente etiquetarse como ODS 1 o ODS 8.

Sobre el efecto del desbalance: es menor de lo que se podria esperar gracias a `class_weight='balanced'`. La clase mas pequena, ODS 12 (62 textos de test), obtiene F1=0.84, mientras que ODS 8, con casi el doble de soporte, es la peor de todas (F1=0.59). La causa dominante de error es la **superposicion tematica del vocabulario**, no el tamano de la clase.

9. Tabla resumen de decisiones (para responder "por que...")

Decision	Alternativa descartada	Razon
F1 macro como metrica de seleccion	Accuracy	Las clases estan desbalanceadas (312-1080); accuracy favorece a las clases grandes
TruncatedSVD (LSA)	PCA	PCA centra los datos y densifica la matriz; inviable con ~26 000 columnas dispersas
15 componentes para interpretar	10 o 20	10 mezcla ODS tematicamente cercanos; 20 produce componentes finales dificiles de leer
300 componentes para clasificar	15 (mismas del LSA interpretativo)	Con 15, F1 macro cae a 0.715: insuficiente para separar 16 clases; se trata como hiperparametro y se busca en {100,200,300}
LinearSVC	LogisticRegression	F1 macro CV ligeramente mejor (0.849 vs 0.844) y ~2x mas rapido; se acepta perder predict_proba
Sin stemming/lematizacion	Aplicar stemming	Corpus formal y homoganeo; se prioriza interpretabilidad de topicos y coeficientes
class_weight='balanced'	Sobre/submuestreo (SMOTE, etc.)	Corrige el desbalance sin duplicar ni descartar texto ni inventar datos sinteticos
Etiqueta unica por texto (top-1)	Clasificacion multi-etiqueta	El dataset trae una sola etiqueta por texto; se mitiga mostrando top-3 por margen
Modelos lineales unicamente	Arboles/boosting/redes neuronales	En texto de alta dimension las clases son casi linealmente separables; el enunciado pide BOW+TFIDF+reduccion de dimensionalidad
Notebook autocontenido (sin paquete src/)	Modulos src/ importados	Restriccion de la plataforma de entrega: solo se pueden subir 2 archivos (notebook y streamlit_app.py)

10. Limitaciones reconocidas y mejoras futuras

10.1 Limitaciones

- **ODS 17 ausente:** el modelo solo conoce 16 clases; cualquier texto sobre alianzas/cooperacion se clasificara forzosamente en algun otro ODS.
- **Etiqueta unica por texto:** muchos textos tocan varios ODS a la vez; el problema real es mas bien multi-etiqueta. El ranking por margen de decision mitiga esto pero no lo resuelve.
- **Dominio del corpus:** textos formales (informes, articulos) traducidos automaticamente. Textos coloquiales de participacion ciudadana (el caso de uso real del UNFPA) tendran vocabulario distinto y el desempeno podria bajar.
- **Bolsa de palabras ignora el orden y la negacion** de las palabras; los bigramas lo compensan solo parcialmente.

- El grid se limitó a 300 componentes y dos algoritmos por costo computacional; la tendencia observada sugiere que 400-500 componentes acercarian aun mas el F1 a la referencia sin reduccion (0.870).

10.2 Mejoras futuras propuestas

- Evaluar 400-500 componentes SVD y una busqueda mas amplia de C, min_df y ngram_range.
- Calibrar probabilidades con CalibratedClassifierCV para que la app muestre confianzas reales en vez de un margen normalizado.
- Reformular como problema multi-etiqueta si se dispone de anotaciones multiples por texto.
- Probar lematizacion con spaCy (es_core_news_md) y comparar el costo en interpretabilidad frente a la posible ganancia en generalizacion.
- Comparar contra embeddings de frases multilingues (p. ej. sentence-transformers) como representacion alternativa a TF-IDF + LSA.

11. Preguntas probables de sustentacion (con respuesta corta)

Por que TF-IDF y no solo conteo de palabras?

Porque el conteo puro sobrepondera palabras frecuentes en todo el idioma. TF-IDF reduce el peso de terminos comunes en el corpus (IDF bajo) y resalta los que son frecuentes en un documento pero raros en general, que son justo los que caracterizan a un ODS.

Que es LSA y para que sirve aqui?

Es aplicar SVD truncado a la matriz TF-IDF para proyectar los documentos a un espacio de menor dimension cuyas direcciones (componentes) son combinaciones de terminos interpretables como topicos. Aqui se usa para dos cosas distintas: interpretar topicos (15 componentes) y como paso de reduccion de dimensionalidad antes de clasificar (100-300).

Por que usan el mismo SVD para dos cosas distintas con distinto numero de componentes?

Porque interpretar y clasificar tienen objetivos distintos: pocas componentes dan ejes legibles pero pierden granularidad (F1 cae a 0.715 con 15); muchas dan mejor separacion de clases pero dejan de ser interpretables individualmente. El notebook lo demuestra numericamente antes de fijar cada valor.

Por que F1 macro y no accuracy?

Porque las clases estan desbalanceadas (312 a 1080 textos). F1 macro pesa cada clase por igual sin importar su tamano, que es el criterio correcto quando importa acertar tanto en ODS pequenos como grandes.

Por que LinearSVC y no Random Forest o una red neuronal?

El enunciado exige una solucion basada en BOW+TF-IDF+reduccion de dimensionalidad. Ademias, en espacios de texto de alta dimension las clases suelen ser casi linealmente separables: los modelos no lineales rara vez mejoran el resultado y cuestan mucho mas computo. Entre los lineales, LinearSVC gana a LogisticRegression en F1 macro CV y en velocidad.

Como evitaron la fuga de datos (data leakage)?

El split train/test se hizo antes de ajustar cualquier transformacion. TF-IDF, SVD y el clasificador viven en un unico Pipeline de scikit-learn que se reajusta completo en cada fold de la validacion cruzada, asi que el vocabulario y el IDF nunca ven datos de validacion o de test durante el entrenamiento.

Que significa que LinearSVC no tenga predict_proba?

LinearSVC optimiza un margen geometrico, no una probabilidad. Para mostrar un ranking de ODS candidatos se usa decision_function (el margen de cada clase), y se le llama explicitamente "puntuacion relativa", no probabilidad, para no sugerir una calibracion que el modelo no tiene.

Por que el modelo falla mas en ODS 8, 10 y 9?

Porque comparten vocabulario (empleo, ingresos, mercado laboral, crecimiento) con ODS 1 y entre si. El propio LSA ya mostraba estos ODS mezclados en las mismas componentes o en polos de un mismo contraste, antes incluso de entrenar el clasificador. No es un problema de tamano de clase: ODS 12, mas pequeno, tiene mejor F1 que ODS 8.

Como se validaria este modelo en produccion, con texto de ciudadanos (no informes formales)?

No deberia asumirse el mismo desempeno: el corpus de entrenamiento es texto formal traducido automaticamente. Habria que evaluar sobre una muestra real de ese dominio antes de confiar en las metricas actuales; es una limitacion explicita del proyecto.

Por que no usaron embeddings (Word2Vec, BERT, sentence-transformers)?

El enunciado pide expresamente resolver el problema con bolsa de palabras + TF-IDF + LSA. Se deja como mejora futura comparar con embeddings de frases como representacion alternativa.

Que garantiza que la app Streamlit use exactamente el mismo modelo que el notebook?

La app carga el mismo objeto Pipeline serializado con joblib (incluye TF-IDF, SVD, Normalizer y el clasificador ya entrenados). La limpieza de texto (clean_text) esta registrada como preprocessor dentro del TfidfVectorizer, asi que no existe ningun paso manual en la app que pueda procesar el texto de forma distinta al entrenamiento.

Por que 5 folds y no 10, por ejemplo?

Es el estandar practico que balancea estabilidad de la estimacion contra costo computacional: con 90 combinaciones de hiperparametros, cada fold adicional multiplica el tiempo total de la busqueda. La desviacion estandar entre folds resulto baja (~0.005), senal de que la estimacion ya es estable con 5.

12. Glosario rapido

Termino	Definicion
TF-IDF	Term Frequency - Inverse Document Frequency. Ponderacion que resalta terminos frecuentes en un documento pero raros en el corpus.
Bolsa de palabras (BoW)	Representacion de un texto como un vector de frecuencias de terminos, sin considerar el orden de las palabras.
n-grama	Secuencia de n palabras consecutivas usada como unidad del vocabulario (unigrama = 1 palabra, bigrama = 2 palabras consecutivas).
LSA (Latent Semantic Analysis)	Tecnica que aplica SVD truncado a una matriz termino-documento para descubrir dimensiones semanticas latentes ("temas").
SVD truncado (TruncatedSVD)	Descomposicion en valores singulares que conserva solo las k direcciones de mayor varianza; a diferencia de PCA, no centra los datos, por lo que funciona sobre matrices dispersas.
Componente (en LSA)	Cada una de las k direcciones resultantes del SVD; es una combinacion lineal de terminos, interpretable como un tema.
Normalizer L2	Transformacion que reescala cada vector para que tenga norma euclidiana 1, haciendo que el producto punto entre vectores equivalga a similitud coseno.
class_weight='balanced'	Opcion de scikit-learn que pondera la funcion de perdida de forma inversamente proporcional a la frecuencia de cada clase.
F1 macro	Promedio del F1 de cada clase con igual peso, independientemente de su tamano; adecuado para datasets desbalanceados.
GridSearchCV	Busqueda exhaustiva de hiperparametros que evalua cada combinacion mediante validacion cruzada y selecciona la de mejor metrica promedio.
StratifiedKFold	Particion en k folds para validacion cruzada que conserva la proporcion de cada clase en cada fold.
decision_function	Metodo de los clasificadores lineales de margen (como LinearSVC) que devuelve la distancia con signo al hiperplano de cada clase, sin ser una probabilidad.
Data leakage (fuga de datos)	Situacion en la que informacion del conjunto de test o de validacion influye, directa o indirectamente, en el entrenamiento, inflando las metricas.
Pipeline (scikit-learn)	Objeto que encadena varias transformaciones y un estimador final de modo que se ajustan y aplican como una sola unidad, evitando fugas de datos.

Fuentes de este documento: notebook ejecutado (notebooks/microproyecto2_ods.ipynb), docs/bitacora_desarrollo.md y CHECKLIST.md del repositorio. Cifras y tablas corresponden a la ultima ejecucion registrada (2026-09-18/19); si se vuelve a ejecutar el notebook con cambios, conviene regenerar este resumen.